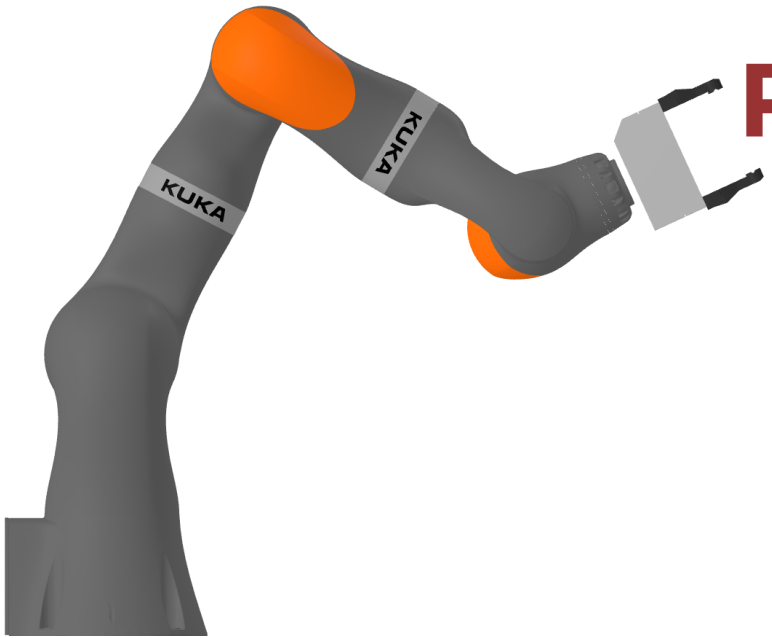


Reinforcement Learning

(Part 2)

MIT 6.4210/2:

Robotic Manipulation

Fall 2024, Lecture 19

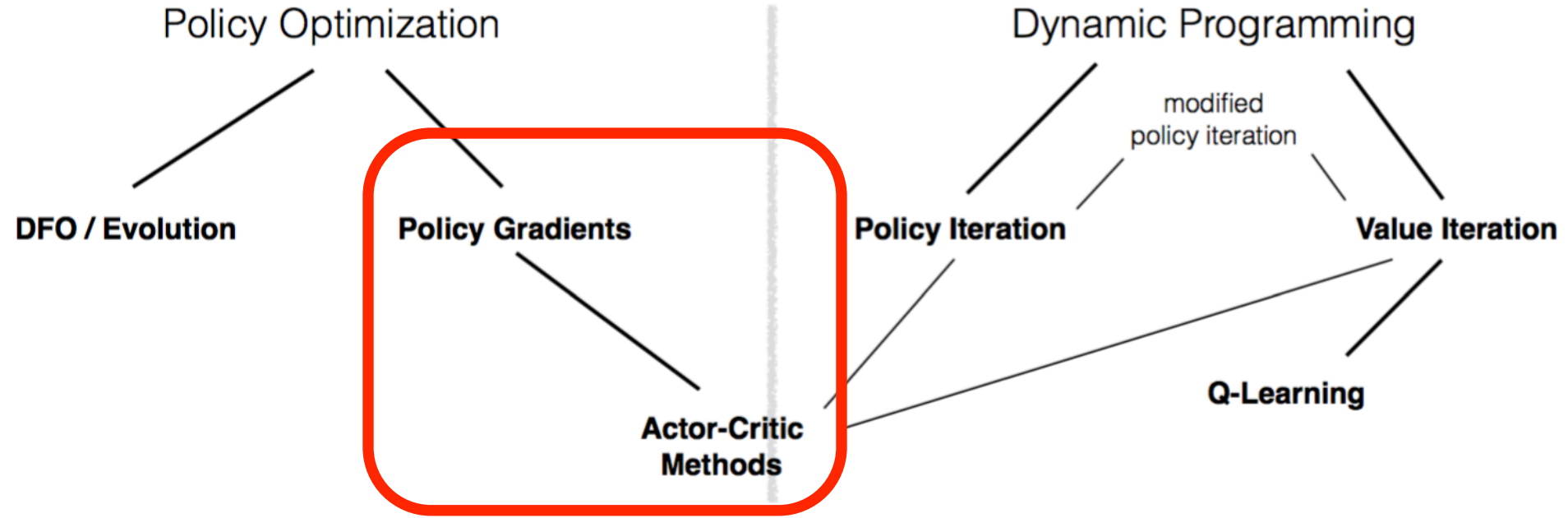
Massachusetts Institute of Technology

Policy Gradients

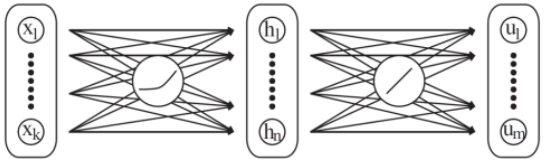
Pieter Abbeel – OpenAI / UC Berkeley / Gradescope

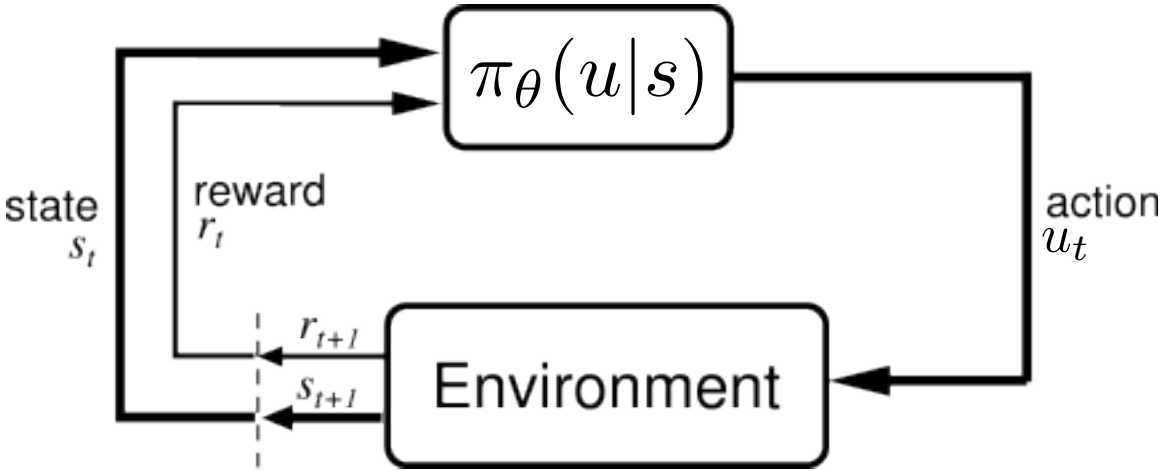
Slides authored with John Schulman and Xi (Peter) Chen

Policy Optimization in the RL Landscape



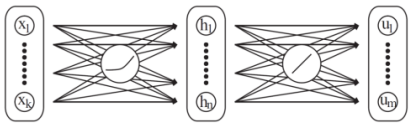
Policy Optimization

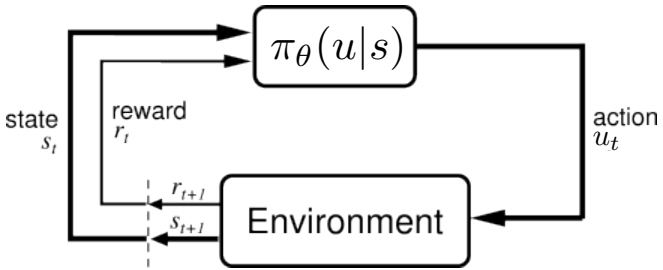




[Figure source: Sutton & Barto, 1998]

Policy Optimization





- Consider control policy parameterized by parameter vector θ

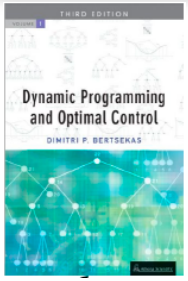
$$\max_\theta \quad \mathrm{E}[\sum_{t=0}^{H} R(s_t)|\pi_\theta]$$

- Stochastic policy class (smooths out the problem):

$\pi_\theta(u|s)$: probability of action u in state s

[Figure source: Sutton & Barto, 1998]

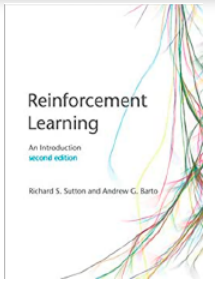
Terminology: State Space Model



OC/DP

environment
controlled plant
dynamical system

AI/RL



State and Control space: $\mathcal{S}, \mathcal{U}$

State: $x_k \in \mathcal{S}, k = 0, 1, \ldots$

Control: $u_k \in \mathcal{U}, k = 0, 1, \ldots$

Dynamical System:

$$x_{k+1} = f(x_k, u_k)$$

$$x_{k+1} = f(x_k, u_k, w_k)$$ stochastic

Output/observation (feature):

$$y_k = h(x_k, u_k) + n_k$$ stochastic

State and Action space: $\mathcal{S}, \mathcal{A}$

State: $s_t \in \mathcal{S}, t = 0, 1, \ldots$

Action: $a_t \in \mathcal{A}, t = 0, 1, \ldots$

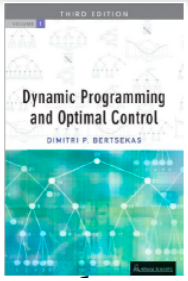
MDP Transition (or simulation):

$$\mathcal{T}_{ijk} = p(s_{t+1} = i \mid s_t = j, a_t = k)$$

Observation (feature):

$$p(o_t \mid s_t)$$

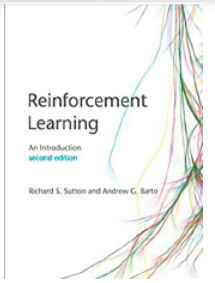
Terminology: Optimization Objective



OC/DP

policy
control
decision

AI/RL



Cost: $g(x_k, u_k) \in \mathbb{R}$

Total cost function:

$$J(x_0; u_0, \ldots, u_N) = \sum_{k=0}^{N} g(x_k, u_k)$$

Control law: $u(x_k);\ u^*(x_k)$

$$x_{k+1} = f(x_k, u(x_k))$$
$$u_{k+1} = u(x_{k+1})$$

Value function (minimal cost to go):

$$J^*(x_0) = \min_{u(\cdot)} \sum_{k=0}^{N} g(x_k, u(x_k, k))$$

Reward: $r(s_t, a_t) \in \mathbb{R}$

Total reward (return):

$$J(s_1; a_1, \ldots, a_T) = \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}[r(s_t, a_t)]$$

Policy: $\pi(a_t \mid s_t);\ \pi^\star(a_t \mid s_t)$

$$p((s_{t+1}, a_{t+1}) \mid (s_t, a_t)) =$$
$$p(s_{t+1} \mid s_t, a_t)\pi(a_{t+1} \mid s_{t+1})$$

Value function (maximal return):

$$V^\star(s_1) = \max_{\pi(\cdot)} \frac{1}{T} \sum_{t=1}^{T} \mathbb{E}_\pi[r(s_t, a_t)]$$

$$\min_{\alpha} E\left[\sum_{n=0}^{N} \ell(\mathbf{x}[n], \mathbf{u}[n])\right] \tag{1}$$

where the random variables are drawn from probability densities

$$\begin{aligned}
\mathbf{x}[0] &\sim p_0(\mathbf{x}), \\
\mathbf{x}[n] &\sim p(\mathbf{x}[n]|\mathbf{x}[n-1], \mathbf{u}[n-1]), \\
\mathbf{u}[n] &\sim p_\alpha(\mathbf{u}[n]|\mathbf{x}[n]).
\end{aligned}$$

Why Policy Optimization

- Often π can be simpler than Q or V
  - E.g., robotic grasp
- V: doesn't prescribe actions
  - Would need dynamics model (+ compute 1 Bellman back-up)
- Q: need to be able to efficiently solve $\arg\max_u Q_\theta(s, u)$
  - Challenge for continuous / high-dimensional action spaces*

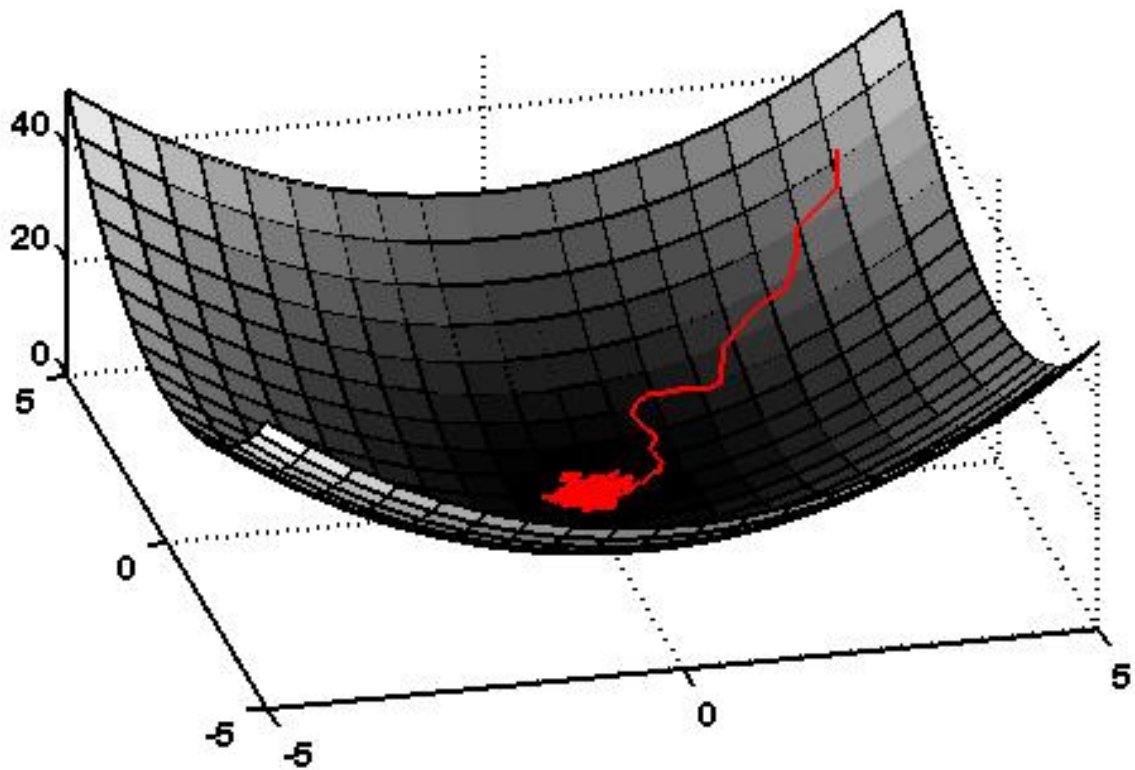
*some recent work (partially) addressing this:

NAF: Gu, Lillicrap, Sutskever, Levine ICML 2016

Input Convex NNs: Amos, Xu, Kolter arXiv 2016

Deep Energy Q: Haarnoja, Tang, Abbeel, Levine, ICML 2017

	Policy Optimization	Dynamic Programming
Conceptually:	Optimize what you care about	Indirect, exploit the problem structure, self-consistency
Empirically:	More compatible with rich architectures (including recurrence) More versatile More compatible with auxiliary objectives	More compatible with exploration and off-policy learning More sample-efficient when they work


40
20
0
5
0
-5
-5
0
5

Likelihood Ratio Policy Gradient

We let τ denote a state-action sequence $s_0, u_0, \ldots, s_H, u_H$. We overload notation: $R(\tau) = \sum_{t=0}^{H} R(s_t, u_t)$.

$$U(\theta) = \mathrm{E}[\sum_{t=0}^{H} R(s_t, u_t); \pi_\theta] = \sum_{\tau} P(\tau; \theta) R(\tau)$$

In our new notation, our goal is to find θ:

$$\max_{\theta} U(\theta) = \max_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau)$$

Likelihood Ratio Policy Gradient

$$U(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\nabla_\theta U(\theta) = \nabla_\theta \sum_{\tau} P(\tau;\theta) R(\tau)$$

[Aleksandrov, Sysoyev, & Shemeneva, 1968]
[Rubinstein, 1969]
[Glynn, 1986]
[Reinforce, Williams 1992]
[GPOMDP, Baxter & Bartlett, 2001]

Likelihood Ratio Policy Gradient

$$U(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned} \nabla_\theta U(\theta) &= \nabla_\theta \sum_{\tau} P(\tau;\theta) R(\tau) \\ &= \sum_{\tau} \nabla_\theta P(\tau;\theta) R(\tau) \end{aligned}$$

[Aleksandrov, Sysoyev, & Shemeneva, 1968]
[Rubinstein, 1969]
[Glynn, 1986]
[Reinforce, Williams 1992]
[GPOMDP, Baxter & Bartlett, 2001]

Likelihood Ratio Policy Gradient

$$U(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned}
\nabla_\theta U(\theta) &= \nabla_\theta \sum_{\tau} P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) R(\tau)
\end{aligned}$$

[Aleksandrov, Sysoyev, & Shemeneva, 1968]
[Rubinstein, 1969]
[Glynn, 1986]
[Reinforce, Williams 1992]
[GPOMDP, Baxter & Bartlett, 2001]

Likelihood Ratio Policy Gradient

$$U(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned}
\nabla_\theta U(\theta) &= \nabla_\theta \sum_{\tau} P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} R(\tau)
\end{aligned}$$

[Aleksandrov, Sysoyev, & Shemeneva, 1968]
[Rubinstein, 1969]
[Glynn, 1986]
[Reinforce, Williams 1992]
[GPOMDP, Baxter & Bartlett, 2001]

Likelihood Ratio Policy Gradient

$$U(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned}
\nabla_\theta U(\theta) &= \nabla_\theta \sum_{\tau} P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} R(\tau) \\
&= \sum_{\tau} P(\tau;\theta) \nabla_\theta \log P(\tau;\theta) R(\tau)
\end{aligned}$$

[Aleksandrov, Sysoyev, & Shemeneva, 1968]
[Rubinstein, 1969]
[Glynn, 1986]
[Reinforce, Williams 1992]
[GPOMDP, Baxter & Bartlett, 2001]

Likelihood Ratio Policy Gradient

$$U(\theta) = \sum_{\tau} P(\tau;\theta) R(\tau)$$

Taking the gradient w.r.t. θ gives

$$\begin{aligned}
\nabla_\theta U(\theta) &= \nabla_\theta \sum_{\tau} P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) R(\tau) \\
&= \sum_{\tau} P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} R(\tau) \\
&= \sum_{\tau} P(\tau;\theta) \nabla_\theta \log P(\tau;\theta) R(\tau)
\end{aligned}$$

Approximate with the empirical estimate for m sample paths under policy π_θ:

$$\nabla_\theta U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)};\theta) R(\tau^{(i)})$$

[Aleksandrov, Sysoyev, & Shemeneva, 1968]
[Rubinstein, 1969]
[Glynn, 1986]
[Reinforce, Williams 1992]
[GPOMDP, Baxter & Bartlett, 2001]

Likelihood Ratio Gradient: Validity

$$\nabla U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

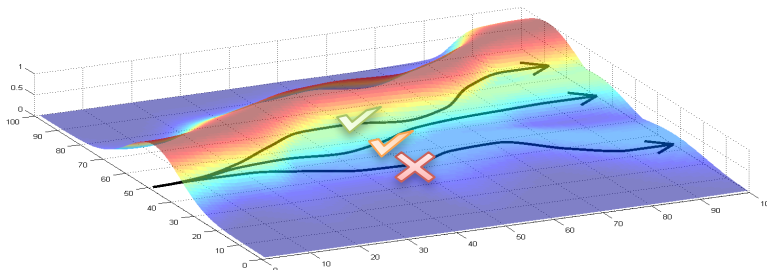
- Valid even when
  - R is discontinuous and/or unknown
  - Sample space (of paths) is a discrete set



Likelihood Ratio Gradient: Intuition

$$\nabla U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

- Gradient tries to:
  - Increase probability of paths with positive R
  - Decrease probability of paths with negative R



! Likelihood ratio changes probabilities of experienced paths, does not try to change the paths (<-> Path Derivative)

Let's Decompose Path into States and Actions

$$\nabla_\theta \log P(\tau^{(i)}; \theta) = \nabla_\theta \log \left[\prod_{t=0}^{H} \underbrace{P(s_{t+1}^{(i)} | s_t^{(i)}, u_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_\theta(u_t^{(i)} | s_t^{(i)})}_{\text{policy}} \right]$$

Let's Decompose Path into States and Actions

$$\nabla_\theta \log P(\tau^{(i)};\theta) = \nabla_\theta \log \left[\prod_{t=0}^{H} \underbrace{P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_\theta(u_t^{(i)}|s_t^{(i)})}_{\text{policy}} \right]$$

$$= \nabla_\theta \left[\sum_{t=0}^{H} \log P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)}) + \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)}) \right]$$

Let's Decompose Path into States and Actions

$$\nabla_\theta \log P(\tau^{(i)};\theta) = \nabla_\theta \log \left[\prod_{t=0}^{H} \underbrace{P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_\theta(u_t^{(i)}|s_t^{(i)})}_{\text{policy}} \right]$$

$$= \nabla_\theta \left[\sum_{t=0}^{H} \log P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)}) + \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)}) \right]$$

$$= \nabla_\theta \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)})$$

Let's Decompose Path into States and Actions

$$\nabla_\theta \log P(\tau^{(i)};\theta) = \nabla_\theta \log \left[\prod_{t=0}^{H} \underbrace{P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)})}_{\text{dynamics model}} \cdot \underbrace{\pi_\theta(u_t^{(i)}|s_t^{(i)})}_{\text{policy}} \right]$$

$$= \nabla_\theta \left[\sum_{t=0}^{H} \log P(s_{t+1}^{(i)}|s_t^{(i)},u_t^{(i)}) + \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)}) \right]$$

$$= \nabla_\theta \sum_{t=0}^{H} \log \pi_\theta(u_t^{(i)}|s_t^{(i)})$$

$$= \sum_{t=0}^{H} \underbrace{\nabla_\theta \log \pi_\theta(u_t^{(i)}|s_t^{(i)})}_{\text{no dynamics model required!!}}$$

Likelihood Ratio Gradient Estimate

The following expression provides us with an unbiased estimate of the gradient, and we can compute it without access to a dynamics model:

$$\hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

Here:

$$\nabla_\theta \log P(\tau^{(i)}; \theta) = \sum_{t=0}^{H} \underbrace{\nabla_\theta \log \pi_\theta(u_t^{(i)} | s_t^{(i)})}_{\text{no dynamics model required!!}}$$

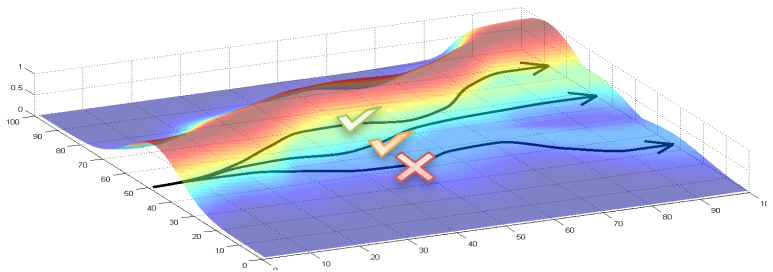
Unbiased means:

$$\mathrm{E}[\hat{g}] = \nabla_\theta U(\theta)$$

Likelihood Ratio Gradient: Intuition

$$\nabla U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

- Gradient tries to:
  - Increase probability of paths with positive R
  - Decrease probability of paths with negative R



! Likelihood ratio changes probabilities of experienced paths, does not try to change the paths (<-> Path Derivative)

1D Illustration (Fixed Variance)

For a single state (drop s) with scalar action and variance σ^2:

$$\pi(a;\mu,\sigma^2) = \mathcal{N}(a;\mu,\sigma^2), \qquad \frac{\partial}{\partial \mu}\log \pi(a;\mu,\sigma^2) = \frac{a-\mu}{\sigma^2}.$$

Using a scalar advantage signal $\widehat{A}(a)$,

$$\mu \leftarrow \mu + \alpha \frac{a-\mu}{\sigma^2}\,\widehat{A}(a).$$

Taking expectation over $a \sim \mathcal{N}(\mu,\sigma^2)$ recovers the true gradient direction:

$$\nabla_\mu J \;=\; \mathbb{E}\left[\frac{a-\mu}{\sigma^2}\,A(a)\right],$$

so the Monte Carlo update above is an unbiased stochastic gradient ascent step (up to variance-reduction baselines).

Likelihood Ratio Gradient Estimate

- As formulated thus far: unbiased but very noisy
- Fixes that lead to real-world practicality
  - Baseline
  - Temporal structure
  - Also: KL-divergence trust region / natural gradient (= general trick, equally applicable to perturbation analysis and finite differences)

Likelihood Ratio Gradient: Baseline

$$\nabla U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta) R(\tau^{(i)})$$

→ Consider baseline b: $\nabla U(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)}; \theta)(R(\tau^{(i)}) - b)$

still unbiased!
[Williams 1992]

$$\begin{aligned}
&\mathbb{E}\left[\nabla_\theta \log P(\tau;\theta) b\right] \\
&= \sum_\tau P(\tau;\theta) \nabla_\theta \log P(\tau;\theta) b \\
&= \sum_\tau P(\tau;\theta) \frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} b \\
&= \sum_\tau \nabla_\theta P(\tau;\theta) b \\
&= \nabla_\theta \left(\sum_\tau P(\tau) b \right) = b \nabla_\theta \left(\sum_\tau P(\tau) \right) = b \times 0
\end{aligned}$$

OK as long as baseline doesn't depend on action in logprob(action)

Likelihood Ratio and Temporal Structure

- Current estimate:

$$\hat{g} = \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta \log P(\tau^{(i)};\theta)(R(\tau^{(i)}) - b)$$

$$= \frac{1}{m}\sum_{i=1}^{m} \left(\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(u_t^{(i)}|s_t^{(i)})\right)\left(\sum_{t=0}^{H-1} R(s_t^{(i)}, u_t^{(i)}) - b\right)$$

$$= \frac{1}{m}\sum_{i=1}^{m} \left(\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(u_t^{(i)}|s_t^{(i)})\left[\left(\sum_{k=0}^{t-1} R(s_k^{(i)}, u_k^{(i)})\right) + \left(\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)})\right) - b\right]\right)$$

Doesn't depend on $u_t^{(i)}$

Ok to depend on $s_t^{(i)}$

- Removing terms that don't depend on current action can lower variance:

$$\frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(u_t^{(i)}|s_t^{(i)})\left(\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)}) - b(s_t^{(i)})\right)$$

[Policy Gradient Theorem: Sutton et al, NIPS 1999; GPOMDP: Bartlett & Baxter, JAIR 2001; Survey: Peters & Schaal, IROS 2006]

Baseline Choices

- Good choice for b?
  - Constant baseline: $b = \mathbb{E}\left[R(\tau)\right] \approx \frac{1}{m}\sum_{i=1}^{m} R(\tau^{(i)})$
  - Optimal Constant baseline: $b = \frac{\sum_i \left(\nabla_\theta \log P(\tau^{(i)};\theta)\right)^2 R(\tau^{(i)})}{\sum_i \left(\nabla_\theta \log P(\tau^{(i)};\theta)\right)^2}$
  - Time-dependent baseline: $b_t = \frac{1}{m}\sum_{i=1}^{m}\sum_{k=t}^{H-1} R(s_k^{(i)}, u_k^{(i)})$
  - State-dependent expected return:

$$b(s_t) = \mathbb{E}\left[r_t + r_{t+1} + r_{t+2} + \ldots + r_{H-1}\right] = V^\pi(s_t)$$

→ Increase logprob of action proportionally to how much its returns are better than the expected return under the current policy

[See: Greensmith, Bartlett, Baxter, JMLR 2004 for variance reduction techniques.]

Estimation of V^π

$$\frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{H-1}\nabla_\theta \log \pi_\theta(u_t^{(i)}|s_t^{(i)})\left(\sum_{k=t}^{H-1} R(s_k^{(i)},u_k^{(i)}) - V^\pi(s_k^{(i)})\right)$$

How to estimate?

- Init $V^\pi_{\phi_0}$
  - Collect trajectories $\tau_1,\ldots,\tau_m$
  - Regress against empirical return:

$$\phi_{i+1} \leftarrow \arg\min_\phi \frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{H-1}\left(V_\phi^\pi(s_t^{(i)}) - \left(\sum_{k=t}^{H-1} R(s_k^{(i)},u_k^{(i)})\right)\right)^2$$

Estimation of V^π

- Bellman Equation for V^π

$$V^\pi(s) = \sum_u \pi(u|s) \sum_{s'} P(s'|s,u)[R(s,u,s') + \gamma V^\pi(s')]$$

- Init $V^\pi_{\phi_0}$
  - Collect data {s, u, s', r}
  - Fitted V iteration:

$$\phi_{i+1} \leftarrow \min_\phi \sum_{(s,u,s',r)} \|r + V^\pi_{\phi_i}(s') - V_\phi(s)\|_2^2 + \lambda \|\phi - \phi_i\|_2^2$$

Vanilla Policy Gradient

Algorithm 1 "Vanilla" policy gradient algorithm

Initialize policy parameter θ, baseline b

for iteration=$1, 2, \ldots$ **do**

Collect a set of trajectories by executing the current policy

At each timestep in each trajectory, compute

the *return* $R_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$, and

the *advantage estimate* $\hat{A}_t = R_t - b(s_t)$.

Re-fit the baseline, by minimizing $\|b(s_t) - R_t\|^2$,

summed over all trajectories and timesteps.

Update the policy, using a policy gradient estimate $\hat{g}$,

which is a sum of terms $\nabla_\theta \log \pi(a_t \mid s_t, \theta) \hat{A}_t$

end for

~ [Williams, 1992]

Advanced Policy Gradient Methods: Natural Gradient, TRPO, and More

John Schulman

OpenAI

August 26, 2017

Two Limitations of "Vanilla" Policy Gradient Methods

- Hard to choose stepsizes
  - Input data is nonstationary due to changing policy: observation and reward distributions change
  - Bad step is more damaging than in supervised learning, since it affects visitation distribution
    - Step too far $\rightarrow$ bad policy
    - Next batch: collected under bad policy
    - Can't recover—collapse in performance
- Sample efficiency
  - Only one gradient step per environment sample
  - Dependent on scaling of coordinates

Reducing reinforcement learning to optimization

- Much of modern ML: reduce learning to numerical optimization problem
  - Supervised learning: minimize training error
- RL: how to *use all data so far and compute the best policy*?
  - Q-learning: can (in principle) include all transitions seen so far, however, we're optimizing the wrong objective
  - Policy gradient methods: yes stochastic gradients, but no optimization problem*
  - This lecture: write down an optimization problem that allows you to do a small update to policy π based on data sampled from π (*on-policy* data)

What Loss to Optimize?

- Policy gradients

$$\hat{g} = \hat{\mathbb{E}}_t \Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{A}_t \Big]$$

- Can differentiate the following loss

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \Big[\log \pi_\theta(a_t \mid s_t) \hat{A}_t \Big].$$

but don't want to optimize it too far

- Equivalently differentiate

$$L^{IS}_{\theta_{\text{old}}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t \right].$$

at $\theta = \theta_{\text{old}}$, state-actions are sampled using θ_{old}. (IS = importance sampling)

Just the chain rule: $\nabla_\theta \log f(\theta)\big|_{\theta_{\text{old}}} = \frac{\nabla_\theta f(\theta)\big|_{\theta_{\text{old}}}}{f(\theta_{\text{old}})} = \nabla_\theta \Big(\frac{f(\theta)}{f(\theta_{\text{old}})} \Big)\Big|_{\theta_{\text{old}}}$

Surrogate Loss: Importance Sampling Interpretation

- Importance sampling interpretation

$$
\begin{aligned}
&\mathbb{E}_{s_t \sim \pi_{\theta_{\text{old}}}, a_t \sim \pi_\theta} \left[A^\pi(s_t, a_t) \right] \\
&= \mathbb{E}_{s_t \sim \pi_{\theta_{\text{old}}}, a_t \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} A^{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right] && \text{(importance sampling)} \\
&= \mathbb{E}_{s_t \sim \pi_{\theta_{\text{old}}}, a_t \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \hat{A}_t \right] && \text{(replace } A^\pi \text{ with estimator)} \\
&= L^{IS}_{\theta_{\text{old}}}(\theta)
\end{aligned}
$$

- Kakade et al.[1] and S. et al.[2] analyze how L^{IS} approximates the actual performance difference between θ and θ_{old}.
- In practice, L^{IS} is not much different than the logprob version $L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t \mid s_t) \hat{A}_t \right]$, for reasonably small policy changes.

[1] S. Kakade and J. Langford. "Approximately optimal approximate reinforcement learning". *ICML*. 2002.

[2] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel. "Trust Region Policy Optimization". *ICML* (2015).

Trust Region Policy Optimization

- Define the following trust region update:

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] \\ \text{subject to} \quad & \hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]] \leq \delta. \end{aligned}$$

- Also worth considering using a penalty instead of a constraint

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] - \beta\hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]]$$

- Method of Lagrange multipliers: optimality point of δ-constrained problem is also an optimality point of β-penalized problem for some β.
- In practice, δ is easier to tune, and fixed δ is better than fixed β

Trust Region Policy Optimization: Pseudocode

- Pseudocode:

  for iteration$=1, 2, \ldots$ **do**

  Run policy for T timesteps or N trajectories

  Estimate advantage function at all timesteps

$$\begin{aligned} &\underset{\theta}{\text{maximize}} \sum_{n=1}^{N} \frac{\pi_\theta(a_n \mid s_n)}{\pi_{\theta_{\text{old}}}(a_n \mid s_n)} \hat{A}_n \\ &\text{subject to} \quad \overline{\mathrm{KL}}_{\pi_{\theta_{\text{old}}}}(\pi_\theta) \leq \delta \end{aligned}$$

  end for

- Can solve constrained optimization problem efficiently by using conjugate gradient
- Closely related to natural policy gradients (Kakade, 2002), natural actor critic (Peters and Schaal, 2005), REPS (Peters et al., 2010)

Review

- Suggested optimizing surrogate loss L^{PG} or L^{IS}
- Suggested using KL to constrain size of update
- Corresponds to natural gradient step $F^{-1}g$ under linear quadratic approximation
- Can solve for this step approximately using conjugate gradient method

Limitations of TRPO

- Hard to use with architectures with multiple outputs, e.g. policy and value function (need to weight different terms in distance metric)
- Empirically performs poorly on tasks requiring deep CNNs and RNNs, e.g. Atari benchmark
- CG makes implementation more complicated

Proximal Policy Optimization: KL Penalized Version

- Back to penalty instead of constraint

$$\underset{\theta}{\text{maximize}} \sum_{n=1}^{N} \frac{\pi_\theta(a_n \mid s_n)}{\pi_{\theta_{\text{old}}}(a_n \mid s_n)} \hat{A}_n - C \cdot \overline{\text{KL}}_{\pi_{\theta_{\text{old}}}}(\pi_\theta)$$

- Pseudocode:

  for iteration$=1, 2, \ldots$ **do**
    Run policy for T timesteps or N trajectories
    Estimate advantage function at all timesteps
    Do SGD on above objective for some number of epochs
    If KL too high, increase β. If KL too low, decrease β.
  end for

- $\approx$ same performance as TRPO, but only first-order optimization

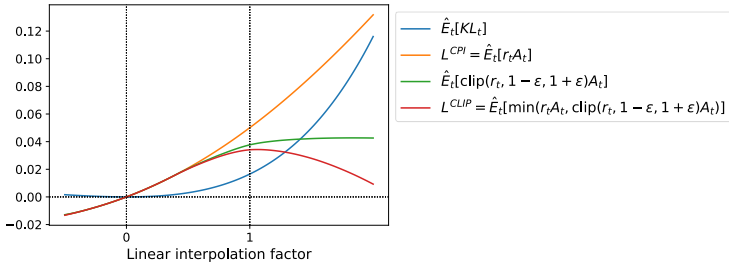
Proximal Policy Optimization: Clipping Objective

- Recall the surrogate objective

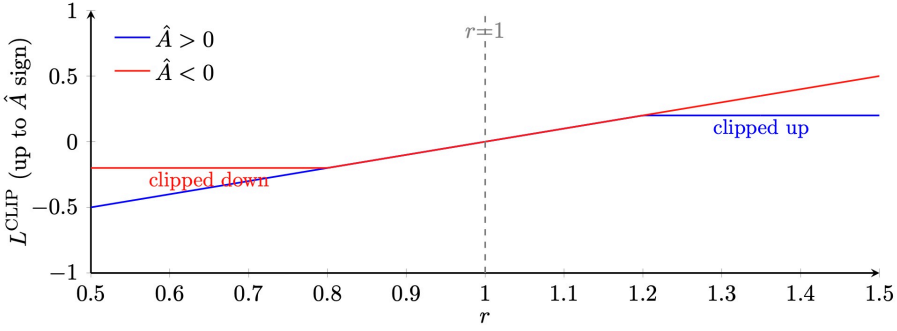
$$L^{IS}(\theta) = \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] = \hat{\mathbb{E}}_t\left[r_t(\theta)\hat{A}_t\right]. \tag{1}$$

- Form a lower bound via clipped importance ratios

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right] \tag{2}$$



- Forms pessimistic bound on objective, can be optimized using SGD


$\hat{A} > 0$
$\hat{A} < 0$
$r=1$
clipped down
clipped up
L^{CLIP} (up to $\hat{A}$ sign)
r
1
0.5
0
−0.5
−1
0.5
0.6
0.7
0.8
0.9
1
1.1
1.2
1.3
1.4
1.5

Proximal Policy Optimization

- Pseudocode:

  for iteration=$1, 2, \ldots$ **do**
  Run policy for T timesteps or N trajectories
  Estimate advantage function at all timesteps
  Do SGD on $L^{CLIP}(\theta)$ objective for some number of epochs
  end for

- A bit better than TRPO on continuous control, much better on Atari
- Compatible with multi-output networks and RNNs

There are two primary variants of PPO: PPO-Penalty and PPO-Clip.

PPO-Penalty approximately solves a KL-constrained update like TRPO, but penalizes the KL-divergence in the objective function instead of making it a hard constraint, and automatically adjusts the penalty coefficient over the course of training so that it's scaled appropriately.

PPO-Clip doesn't have a KL-divergence term in the objective and doesn't have a constraint at all. Instead relies on specialized clipping in the objective function to remove incentives for the new policy to get far from the old policy.

Here, we'll focus only on PPO-Clip (the primary variant used at OpenAI).

Algorithm 1 PPO-Clip

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

for $k = 0, 1, 2, \ldots$ **do**

Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

Compute rewards-to-go $\hat{R}_t$.

Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

end for